Final Project Req. ACC to Mandatory 1

Cover page, including Title, Full names of all students in the group, Group number, Date of delivery.

Shift Happens

Group 11
Members:
Aleksandr Uktamovich Sorokin - also1002@stud.ek.dk
Lucas Levy l'Espagnol Chantelou - luch0002@stud.ek.dk
Jafar Alsufaki - jaal0002@stud.ek.dk
Sebastian Holger Drumm - sedr0001@stud.ek.dk
Viggo Møhring Beck - vibe0002@stud.ek.dk

Date of delivery: 25/04/2026
Repo: https://github.com/Luke3520/shift_happens

Shift Happens **1**
1. Introduction **2**
1.1. System overview and cloud architecture: This section must include: 3
1.2. Explanation of choices for databases and programming languages, and other tools 3
2. Relational database **3**
2.1. Intro to relational databases 3
2.2. Database design 4
2.2.1. Entity/Relationship Model (Conceptual -> Logical -> Physical model) Conceptual Model 4
2.2.2. Normalization process 5
2.3. Physical data model 6
2.3.1. Data types 7
2.3.2. Primary and foreign keys 7
2.3.3. Indexes 7
2.3.4. Constraints and referential integrity 8
2.4. Stored objects – stored procedures / functions, views, triggers, events 8
Functions: 10
Views 11
Triggers **12**
Events 12
2.5. Transactions. Explanation of the structure and implementation of transactions 13

2.6. Auditing. Explanation of the audit structure implemented with triggers..... 13
2.7. Security..... 14
2.7.1. Explanation of users and privileges..... 14
2.7.2. SQL Injection – what is it and how is it dealt with in the project?..... 15
2.8. Description of the CRUD application for RDBMS..... 15
3. Document database..... 15
3.1 Intro to document databases..... 15
3.2 Database design..... 16
3.2.1 Employee Document Design..... 16
Embedded structure includes:..... 16
Key design considerations:..... 17
3.2.2 Shift Document Design..... 17
Embedded structure includes:..... 17
Design of Shift Assignments and Swaps..... 17
3.2.3 Additional Collections..... 18
Design Trade-offs..... 18
Advantages:..... 18
Disadvantages:..... 18
Conclusion of Design Approach..... 19
4. Graph database..... 19
4.1. Intro to graph databases..... 19
4.2. Database design..... 20
4.3. Short descriptions of features like indexes, transactions, PKs, constraints, stored objects (if stored objects are not available, how were they replaced?)..... 21
Indexes..... 21
Transactions..... 22
Primary Keys and Constraints..... 22
Stored Objects and Business Logic..... 23
Constraints and Data Integrity..... 23

1. Introduction

A shift scheduling system, possibly extended with a payroll system to meet database requirements.

We are developing a system similar to Quinyx and Planday. It is not a system designed for a specific company, but rather a broad solution that can be used by many different companies that need shift scheduling across departments and organizations.

1.1. System overview and cloud architecture: This section must include:

• Architecture diagram showing the system deployed in the cloud, and
• Diagram showing the local development deployment using docker-compose.

1.2. Explanation of choices for databases and programming languages, and other tools.

2. Relational database

This chapter describes the design and implementation of the persistence layer for "Shift Happens". As the system handles critical business data such as employee contracts, shift schedules, and payroll data, data integrity and structure are paramount.

The project is structured in phases, where this first phase focuses on the implementation of a relational database (RDBMS). For this purpose, MySQL has been chosen as the database server, and interaction from the application layer is handled through Spring Data JPA. This choice ensures robust handling of the complex relationships that exist between the system's core entities such as employees, departments, and shifts.

2.1. Intro to relational databases

A relational database organizes data into tables (relations) consisting of rows and columns, where each row represents a unique record, and the columns define the attributes for that record.
The choice of a relational database for “Shift Happens” is based on several key factors that align with the systems requirements:

- Structured data: Data in a shift planning system is inherently highly structured. For example, an employee always possesses specific attributes like first_name, email and hire_date. An RDBMS enforces this structure through a fixed schema, ensuring data quality.

- Relationships: The system's logic relies on strong couplings between entities. A shift fx. can not exist without belonging to a department, and an employee_contract must be linked to a specific employee. RDBMS supports these relationships directly through PK & FK, enabling the maintenance of referential integrity.

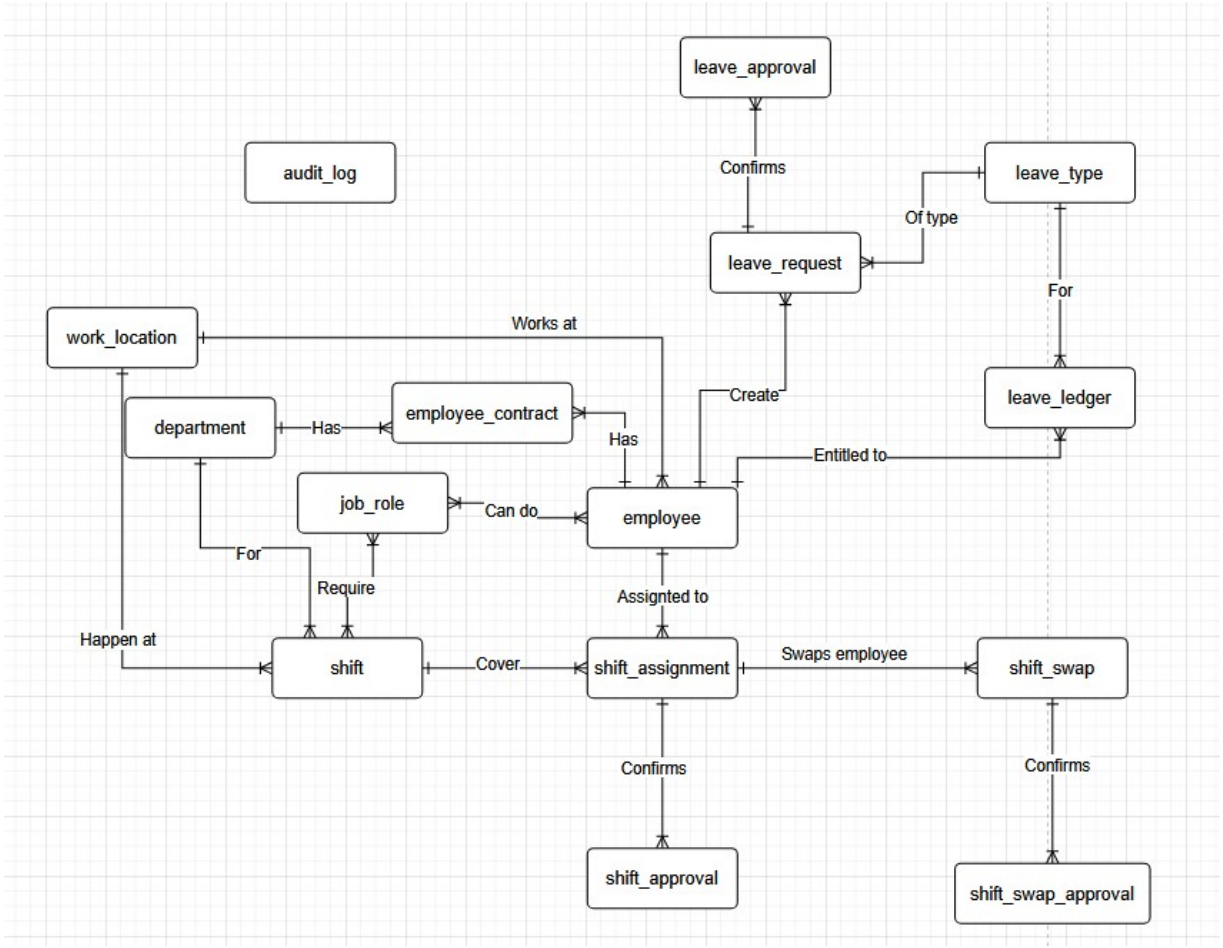
- SQL standard: The use of Structured Query Language (SQL) provides a standardized way to define (DDL) and manipulate (DML) data. This is essential for the implementation of the advanced stored procedures and triggers utilized by the system for tasks such as audit logging and business rule validation.

2.2. Database design

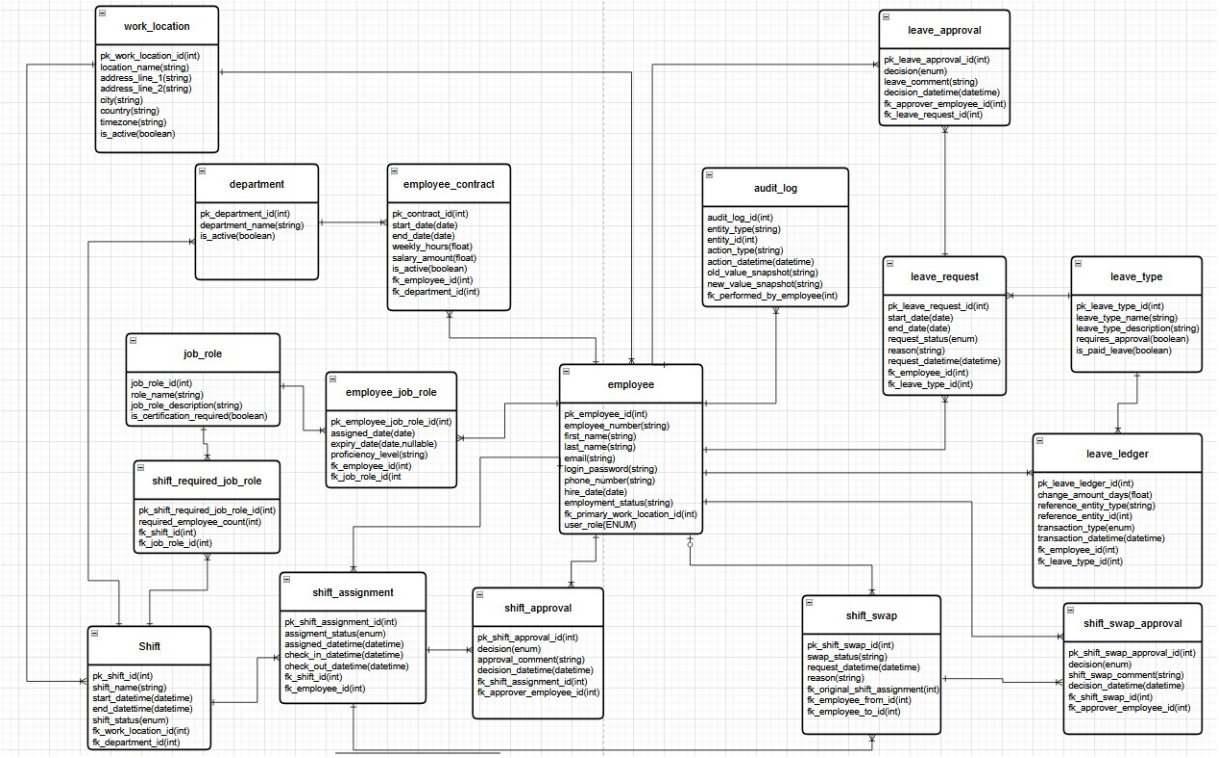
We used draw.io to create our conceptual and logical database models. The models are stored here: https://app.diagrams.net/#G11ves57F6L3OeHWXm3f4J7CuvY37tEr9K%23%7B%22pageId%22%3A%228Iflm39Hw-BSWISrKM0s%22%7D

2.2.1. Entity/Relationship Model (Conceptual -> Logical -> Physical model)

Conceptual Model



Logical Model



2.2.2. Normalization process

During the conceptual design phase, the main domain entities of the system were identified. These entities were then refined in the logical model, where attributes and relationships were defined.

Following the development of the logical model, a normalization review was conducted to ensure that the schema satisfies the first three normal forms. Each table was designed to represent a single entity, with all non-key attributes depending directly on the primary key.

During this process, several many-to-many relationships were identified and resolved through the introduction of join tables. Additionally, some entities were decomposed into smaller, independent tables to eliminate redundancy and prevent update, insert, and delete anomalies.

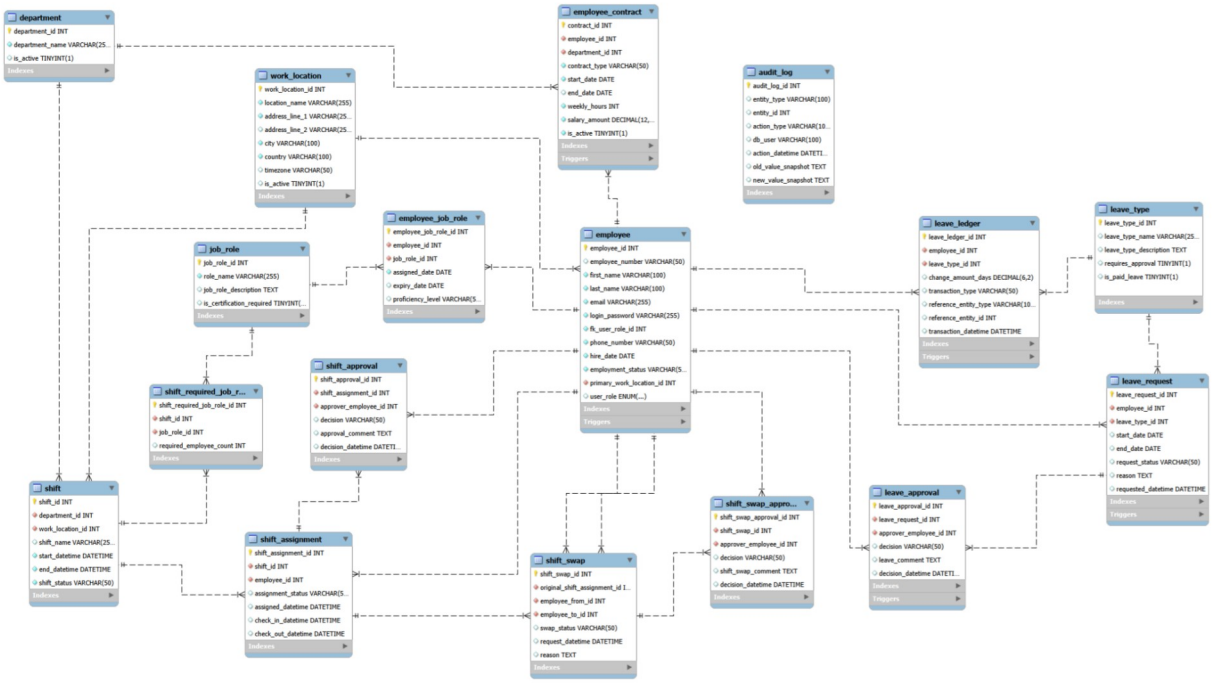
As a result, the final relational schema consists of multiple related tables that may appear structurally similar but represent distinct domain entities. While the schema generally satisfies the Third Normal Form (3NF), a deliberate design decision was made regarding address-related attributes.

In theory, certain address fields (such as postal code and city) may introduce transitive dependencies, as the city can be functionally dependent on the postal code rather than directly on the primary key. However, further normalization—such as extracting postal codes into a separate table—was intentionally avoided. The system does not rely on advanced geographical

logic, and introducing additional tables would increase schema complexity without providing meaningful business value.

This decision represents a controlled denormalization trade-off, prioritizing simplicity and maintainability over strict theoretical normalization. If future business requirements demand geographical validation or location-based logic, the schema can be refactored accordingly.

2.3. Physical data model



The physical data model is the final step in the database design process and builds directly on the logical model. While the logical model defines entities, attributes, and relationships at an abstract level, the physical model translates this structure into a concrete implementation in MySQL.

In this stage, entities are implemented as tables, and attributes are assigned specific data types. Primary keys, foreign keys, and constraints such as `NOT NULL` and `UNIQUE` are defined to ensure data integrity. Additionally, implementation-specific decisions—such as indexing, naming conventions, and data type selection—are applied to optimize performance and maintainability.

The physical model also includes database-level logic such as triggers and stored procedures, supporting business rules and ensuring consistency. Overall, it represents the operational version of the database schema used by the application.

2.3.1. Data types

Although `ENUM` could have been used to strictly restrict possible status values, we deliberately chose `VARCHAR` to maintain flexibility across different organizational configurations. An exception to this is the user_role however, as we don't expect to see many new roles getting made, this would require application level changes anyway, so having it set to an enum is fine. This design allows future extensibility without requiring schema migrations when new status values are introduced. However, we acknowledge that this approach reduces database-level validation and shifts integrity enforcement to the application layer. In a production-scale enterprise system, a more robust solution would likely involve dedicated lookup tables with foreign key constraints to balance flexibility and data integrity.

For free-form descriptions and comments, we chose the `TEXT` data type. This does not introduce performance concerns as long as these fields are not used for filtering or indexing. Since they are intended purely for descriptive content, `TEXT` provides sufficient flexibility without practical drawbacks. If full-text search functionality becomes a requirement in the future, we would introduce `FULLTEXT` indexes or integrate a dedicated search solution, as standard B-tree indexes are not efficient for substring search patterns.

2.3.2. Primary and foreign keys

Primary and foreign keys are used in our database to define relationships between tables and to keep it consistent and clean. Each table has a primary key, such as employee_id, department_id, shift_id and leave_request_id. They uniquely identify each row in the table, whereas foreign keys are used to link related tables together.

This ensures that related data must exist before it can be inserted. For example, a shift cannot reference a department that does not exist, and a leave request cannot be created for a non existing employee.

2.3.3. Indexes

Indexes in SQL allows for faster SELECT queries on chosen columns, this is very useful if we have an idea of what we would like to query in advance. Indexes work by adding pointers for specific values allowing for increased speed when using WHERE clauses.

Although indexes improve query speed for SELECT, it does also come with potential downsides. It will increase storage space used by the table, it will also increase UPDATE,INSERT and DELETE statements as they will subsequently have to update the index as well. Therefore a balance has to be found, by only using indexes on queries we need faster speed for, we can keep the amount down.

Primary, unique and foreign keys get an index added automatically, so we can skip going over those. Apart from those we have added the following indexes with an example situation.

Index	Table	Columns	Situation
idx_leave_ledger_emp_type	leave_ledger	(employee_id, leave_type_id)	Calculate available leave of a specific type for an employee
idx_dep_type_ac	employee_contract	(department_id, contract_type, is_active)	Find active full/part-time contracts/employees for a department
idx_dep_loc	shift	(department_id, work_location_id)	Find shifts at a location in a specific department
idx_paid_approval	leave_type	(leave_type_id, requires_approval, is_paid_leave)	Filtering leave requests that don't require approval and are paid leaves.

Source: MySQL 8.4 Reference Manual :: 10.3 Optimization and Indexes

2.3.4. Constraints and referential integrity

2.4. Stored objects – stored procedures / functions, views, triggers, events

Stored Procedures:
Currently we have create **sp_sumbit_leave_request** as a stored procedure where we make sure to validate an employee has enough leave balance before submitting a leave request. An error message is sent if the employee lacks balance.

```sql
DELIMITER $$

CREATE PROCEDURE sp_submit_leave_request(
    IN p_employee_id INT,
    IN p_leave_type_id INT,
    IN p_start_date DATE,
    IN p_end_date DATE,
    IN p_reason TEXT
)
BEGIN
    DECLARE v_days_requested DECIMAL(6, 2);
    DECLARE v_balance DECIMAL(6, 2);

    SET v_days_requested = DATEDIFF(p_end_date, p_start_date) + 1;

    -- Check balance using our function
    SET v_balance = fn_get_leave_balance(p_employee_id, p_leave_type_id);

    IF v_balance < v_days_requested THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Insufficient leave balance to submit this request';
    END IF;

    INSERT INTO leave_request (employee_id, leave_type_id,
                               start_date, end_date,
                               request_status, reason, requested_datetime)
    VALUES (p_employee_id, p_leave_type_id,
            p_start_date, p_end_date,
            'PENDING', p_reason, NOW());

    SELECT LAST_INSERT_ID() AS new_leave_request_id;
END$$

DELIMITER ;
```

Furthermore we also have one for **sp_assign_employee_to_shift** which assigns an employee to a shift but firstly checks if there's overlapping/conflicting shifts assignment. Raises an error on conflict.

```
DELIMITER $$

CREATE PROCEDURE sp_assign_employee_to_shift(
    IN p_shift_id INT,
    IN p_employee_id INT
)
BEGIN
    DECLARE v_conflict_count INT;
    DECLARE v_shift_start DATETIME;
    DECLARE v_shift_end DATETIME;

    -- Get the shift times
    SELECT start_datetime, end_datetime
    INTO v_shift_start, v_shift_end
    FROM shift
    WHERE shift_id = p_shift_id;

    -- Check for overlapping shift assignments
    SELECT COUNT(*)
    INTO v_conflict_count
    FROM shift_assignment sa
            JOIN shift s ON sa.shift_id = s.shift_id
    WHERE sa.employee_id = p_employee_id
      AND sa.assignment_status IN ('ASSIGNED', 'CONFIRMED')
      AND s.start_datetime < v_shift_end
      AND s.end_datetime > v_shift_start;

    IF v_conflict_count > 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Employee has a conflicting shift assignment';
    END IF;

    INSERT INTO shift_assignment (shift_id, employee_id,
                                  assignment_status, assigned_datetime)
    VALUES (p_shift_id, p_employee_id,
            'ASSIGNED', NOW());

    SELECT LAST_INSERT_ID() AS new_assignment_id;
END$$

DELIMITER ;
```

Functions:

Stored functions are used in the system to encapsulate reusable business logic directly in the database layer. Instead of repeatedly implementing the same calculations in the application code, the logic is centralized in database functions. This improves consistency and ensures that the same rules are applied regardless of how the data is accessed.

One example is the calculation of the remaining leave balance for an employee. The function `fn_get_leave_balance` calculates the current balance by summing all transactions stored in the `leave_ledger` table. The ledger-based design records both accruals and deductions of leave days, and the function dynamically calculates the current balance when needed. This

approach avoids storing redundant data and ensures that the balance always reflects the latest transactions.

Another example is the function `fn_department_headcount`, which returns the number of active employees in a department. The function counts employees whose contracts are marked as active and whose employment status is active. This allows the system to quickly retrieve department headcounts without repeating the same query logic in multiple places.

Both functions are defined as `DETERMINISTIC` and `READS SQL DATA`, meaning that they do not modify data and always return the same result for the same input parameters. Using stored functions for these calculations improves maintainability, reduces duplicated logic in the application layer, and ensures consistent results across the system.

It should also be noted that stored functions are executed for each row when used in queries, which may introduce performance overhead in very large datasets. However, in the context of this system the functions are primarily used for targeted calculations, such as retrieving a single employee's leave balance or department headcount, and therefore do not pose a performance concern.

Views

In our project, we use database views to create simplified data extracts from multiple tables. The purpose of using views is to avoid writing complex queries every time we need to retrieve data. Instead of repeating joins and filtering logic in every query, we centralize this logic inside the database view.

This approach improves performance and development efficiency. By predefining the query logic in a view, we reduce the complexity of the queries executed from the backend. Especially when working with complex joins across multiple tables, views provide a cleaner and more maintainable solution. Additionally, they make it easier to retrieve frequently used data structures without rewriting complicated JOIN statements.

We use views to combine data from the Employees and Shifts tables. These two tables are frequently queried together, both to retrieve an overview of all employees and their assigned shifts, and to fetch shift information for a single employee. Since this is an essential part of the business logic and is frequently accessed by users, it made sense to encapsulate the query logic in a view.

We also use views to combine the Employees and Leave models. In this case, the view aggregates leave-related data into a single structured result set. Compared to the Employees–Shifts view, this one contains more attributes and additional filtering requirements. For example, we implemented multiple GET endpoints that allow filtering not only by ID and general overview, but also by leave status (e.g., Approved, Pending, Rejected). This allows us to easily query leave requests based on their workflow status.

Since we are developing the backend using Java and Spring Boot, we use entity classes and DTOs (Data Transfer Objects) to structure the data. The entity classes represent the full data model, while DTOs are used to expose only the necessary data to the client. This improves scalability and security, as we can easily create new DTOs tailored to specific use cases without exposing sensitive or unnecessary fields. It also ensures that our internal data structure remains decoupled from the API layer.

Triggers

SQL triggers are named database objects consisting of code that run when a certain event occurs for the table they live in. This could be BEFORE/AFTER an INSERT/UPDATE/DELETE statement. They exist on the table-level, so a cross-table trigger is not possible, in that case, a stored procedure is probably a better fit for those use cases.

Here are the triggers for this project and their purpose.

- Ledger entries cannot be deleted (**trg_no_delete_leave_ledger**)
- Ledger change amount cannot be 0 (**trg_validate_ledger_amount**)
- Employee on leave request cannot change (**trg_prevent_employee_change**)
- Password needs to be longer than 8 chars (**trg_validate_employee_ins**)
- Salary needs to be positive (**trg_validate_contract_ins**)
- Weekly hours need to be positive (**trg_validate_contract_ins**)
- Start date cannot be after end date (**trg_validate_contract_ins**)
- Overlapping contracts are not allowed (**trg_no_contract_overlap_ins**)

Besides the above logic based triggers we also have automated audit log triggers for all tables except audit_log that logs any changes into the audit log table. Further info in section 2.6

Source: MySQL 8.4 Reference Manual :: 27.3 Using Triggers

Events

Events are tasks that run according to a schedule, for example every hour, every day or every minute. They're similar to cron jobs on application servers. They are very useful for cleaning up tables, updating time based values, they're also known as temporal triggers.

For our project we have implemented the following events.

evt_auto_complete_shifts
This event fires every day, the purpose of this event is to close any shift that has already ended but still has the 'PLANNED' status. This ensures that no shifts get started after they end, and cleans up any remaining shifts that are not completed.

evt_expire_old_leave_requests
This event also fires once a day, here we check if a leave request has been pending for more than 30 days, if so, we automatically set it to EXPIRED, this does not set it to REJECTED as that would require a managers approval, but simply expires it.

Source: MySQL 9.6 Reference Manual :: 27.5.1 Event Scheduler Overview

2.5. Transactions. Explanation of the structure and implementation of transactions

Transactions in MySQL allows one to lock down tables ensuring that your logic runs and if something fails you can rollback any changes made. This allows one to ensure any concurrent queries don't affect each other.

Below we have a stored procedure which will benefit from the above security.
Leave requests approvals
Situation: Manager approves leave request by employee
Description:
A leave request can be approved by managers, if the approving employee is a manager, the dates are correct, and there's enough leave available in the leave ledger, a leave request can then be approved. This also needs to insert the usage into the leave ledger subsequently.

There are several reasons this needs to be a transaction. For instance, let's say a leave approval gets approved but it doesn't insert a subtraction/usage entry into the leave ledger, this would in theory give the employee a free leave request.

Another reason could be that 2 managers approve the same request at the same time, this could in theory insert 2 subtractive entries into the ledger, which in turn would unfairly subtract double the amount of leave days an employee has for a leave request. For this reason we have wrapped the whole procedure into a transaction and roll back any changes should it fail.

2.6. Auditing. Explanation of the audit structure implemented with triggers

Auditing in the relational database is implemented using database triggers on each table. The purpose of the auditing mechanism is to ensure traceability, accountability, and historical tracking of all data modifications.

Three triggers have been created:

- AFTER INSERT
- BEFORE UPDATE
- BEFORE DELETE

Each trigger automatically inserts a record into the audit_log table whenever a change occurs.

The audit_log table stores all relevant audit information, with the most important aspects being who performed the change, and when the change occurred.

- db_user – the user responsible for the change
- action_datetime – timestamp of the action

For INSERT operations, only the `new_value_snapshot` is stored.
For DELETE operations, only the `old_value_snapshot` is stored.
For UPDATE operations, both old and new snapshots are recorded.

The snapshots are stored using the JSON_OBJECT( ) function, allowing the system to capture a complete state of the row at the time of modification. Sensitive data such as `login_password` is masked and never stored in plaintext in the audit table.

By implementing auditing at the database level using triggers, we ensure that audit logging cannot be bypassed by the application layer. This guarantees consistency and reliability of the audit data regardless of how the database is accessed.

2.7. Security.

2.7.1. Explanation of users and privileges

Database-level security in this project is implemented through a dedicated application user with restricted privileges. Instead of using a highly privileged account, a custom role (app_crud_role) is created and assigned to the application user. This role follows the principle of least privilege by granting only the permissions required for the application to function.

The application user is allowed to perform standard CRUD operations (SELECT, INSERT, UPDATE, DELETE) on core domain tables. However, no Data Definition Language (DDL) permissions (e.g., CREATE, ALTER, DROP) are granted, preventing structural modifications to the database. Certain tables are further restricted: for example, the audit_log table is read-only, and the leave_ledger table only allows SELECT and INSERT operations, ensuring data integrity and preventing unauthorized updates.

We considered implementing multiple database users (e.g., manager-level and employee-level users) to enforce fine-grained access control directly at the database level. However, this was not pursued, as similar restrictions are already enforced in the application layer using Spring Security and user roles.

Having a separate DB user for each application user role introduces some issues as well such as connection pooling, introducing a new connection per role or maybe even per user. Spring

boot works by making a pool of connections tied to a single DB user and uses these efficiently. Introducing more DB users would break this and introduce higher complexity.

Overall, the database user setup provides a secure baseline by limiting direct database access, while application-level security handles more granular authorization logic.

2.7.2. SQL Injection – what is it and how is it dealt with in the project?

SQL Injection is a vulnerability where malicious input is used to manipulate SQL queries, potentially allowing unauthorized access or modification of data. In this project, SQL Injection is mitigated by using Spring Boot's JpaRepository, which relies on parameterized queries and prepared statements. This ensures that user input is treated strictly as data and not executable SQL.

All database interactions are handled through repository methods, and no raw SQL or string concatenation is used when building queries. Input parameters, such as path variables and request bodies, are safely mapped and processed by the framework.

As long as this approach is maintained, the risk of SQL Injection is minimal. However, if native queries or dynamic string-based queries were introduced, additional precautions would be required.

2.8. Description of the CRUD application for RDBMS

• Mainly the data layer – models, repositories, etc. but also briefly mention the other layers like services and controllers.
• Graphical schema of the backend modules (model, repository, service, controller, etc).
• Other relevant topics like transactions, calling the stored procedures, security, integration testing, AI integration

. References: All major design choices, technical claims, and comparisons must be supported by references. Sources must be cited in the text and listed in the References section. References should primarily be official documentation, textbooks, or academic or technical publications. Use standard formats like APA, IEEE, or Harvard.

3. Document database

3.1 Intro to document databases

Document databases, such as MongoDB, are NoSQL databases designed to store and manage data in flexible, JSON-like documents rather than structured tables. Each document represents a self-contained data structure that can include nested objects and arrays, allowing complex relationships to be stored within a single record.

Unlike relational databases, document databases do not rely on rigid schemas or join operations. Instead, they prioritize **flexibility, scalability, and data locality**, meaning related data is often stored together in a single document. This makes document databases particularly suitable for applications where data is frequently read as complete aggregates rather than as individual normalized entities.

In this project, MongoDB is used as an alternative representation of the relational shift management system. The goal is to preserve the same business functionality while adapting the data model to a document-oriented structure. This requires **denormalization of relational tables** into embedded documents, where entities such as employees and shifts become central aggregates containing their related data (e.g. contracts, leave requests, assignments, and approvals).

The main advantages of using a document database in this system include:

- Faster read operations due to reduced need for joins
- Natural mapping to object-oriented application models
- Flexibility in evolving the schema without migrations
- Ability to store hierarchical and nested data structures directly

However, document databases also introduce trade-offs such as data duplication, potential inconsistencies across embedded data, and more complex update operations when deeply nested structures are involved.

3.2 Database design

The MongoDB design for the shift management system is based on two primary **aggregate roots**:

- **Employee**
- **Shift**

These aggregates represent the most frequently accessed and logically independent units in the system. All related data is embedded within these documents to optimize read performance and simplify API responses.

3.2.1 Employee Document Design

The Employee document acts as the central user entity and contains both static employee information and dynamic operational data.

Embedded structure includes:

- **Primary work location snapshot**
- **Contracts (with department information embedded)**

- **Job roles**
- **Leave requests and approvals**
- **Leave ledger (historical balance tracking)**

This design ensures that all information required for HR-related views is available in a single document without requiring cross-collection joins.

Key design considerations:

- Contracts are embedded because they are tightly coupled to the employee lifecycle.
- Leave requests are embedded because they are rarely queried independently of the employee.
- Approvals are nested within leave requests to preserve historical decision context.

3.2.2 Shift Document Design

The Shift document represents operational scheduling data and includes all information required for shift execution and management.

Embedded structure includes:

- **Department snapshot**
- **Work location snapshot**
- **Required job roles**
- **Shift assignments**
- **Shift approvals**
- **Shift swap requests and approvals**

This structure allows the entire shift state, including participants and lifecycle events, to be retrieved in a single query.

Design of Shift Assignments and Swaps

Shift assignments are embedded within the shift document because they are inherently part of the shift lifecycle. Each assignment contains:

- Assigned employee snapshot
- Assignment status and timestamps
- Approval history

Shift swaps are also embedded within assignments, as they are directly related to the assignment lifecycle. Each swap includes:

- Source and target employee snapshots

- Swap status and metadata
- Swap approval history

This results in a deeply nested but logically consistent structure that reflects real-world workflow dependencies.

A separate collection for shift swaps was considered to reduce nesting and document size. However, this would require additional lookups when retrieving shift assignments, increasing query complexity.

Since shift swaps are expected to be relatively infrequent, the embedded approach was chosen to optimize read performance and ensure all shift-related data can be retrieved in a single operation.

3.2.3 Additional Collections

The following supporting collections are modeled as independent documents due to their static or reference-based nature:

- **Department**
- **Work Location**
- **Job Role**
- **User Role**
- **Leave Type**
- **Audit Log**

These collections are referenced or embedded depending on usage context. For example, frequently accessed metadata such as department name or location name is embedded as snapshots in employee and shift documents to optimize read performance and reduce lookup complexity.

Design Trade-offs

The MongoDB model prioritizes **read efficiency and data locality** over strict normalization. This results in the following trade-offs:

Advantages:

- Reduced need for joins and complex queries
- Faster retrieval of complete employee or shift views
- Natural representation of hierarchical business processes
- Simplified API design (single-document responses)

Disadvantages:

- Data duplication across documents (e.g. department names, employee names)
- Potential inconsistencies if reference data changes
- Increased document size due to nested structures
- More complex updates in deeply nested arrays

Conclusion of Design Approach

The MongoDB schema is designed as a **domain-driven aggregate model**, where Employee and Shift act as root entities containing all relevant operational data. This design reflects a typical document database philosophy: optimizing for read performance and application-level data consistency rather than strict relational normalization.

The structure enables efficient backend operations for a shift management system, where most queries require full context of an employee or shift rather than isolated relational fragments.

4. Graph database

4.1. Intro to graph databases

Graph databases are designed to efficiently handle highly connected data by storing relationships directly between entities, rather than relying on join operations as in relational databases. This makes them particularly well-suited for domains where relationships between data are complex and frequently queried.

In a graph database, data is represented using nodes and relationships. Nodes represent entities such as *Employee*, *Shift*, and *Department*, while relationships represent the connections between them, for example *WORKS_AT_Dept* or *ASSIGNED_TO_SHIFT*. Unlike relational databases, where relationships are inferred through foreign keys and joins, graph databases store these connections explicitly, enabling faster and more intuitive data traversal.

Graph databases are optimized for relationship-heavy queries, as they allow direct navigation between connected nodes without the need for expensive join operations. This results in improved performance, especially as the depth and complexity of relationships increase. Query

languages such as Cypher (used in Neo4j) are specifically designed to express these traversals in a clear and efficient way.

Typical use cases for graph databases include social networks, recommendation systems, and systems with complex dependencies between entities. In this project, a graph database is used to support a shift management application. The model enables efficient querying of employee schedules, relationships between shifts and departments, and simplifies operations such as shift swaps, where connections between multiple employees and shifts must be evaluated quickly.

4.2. Database design

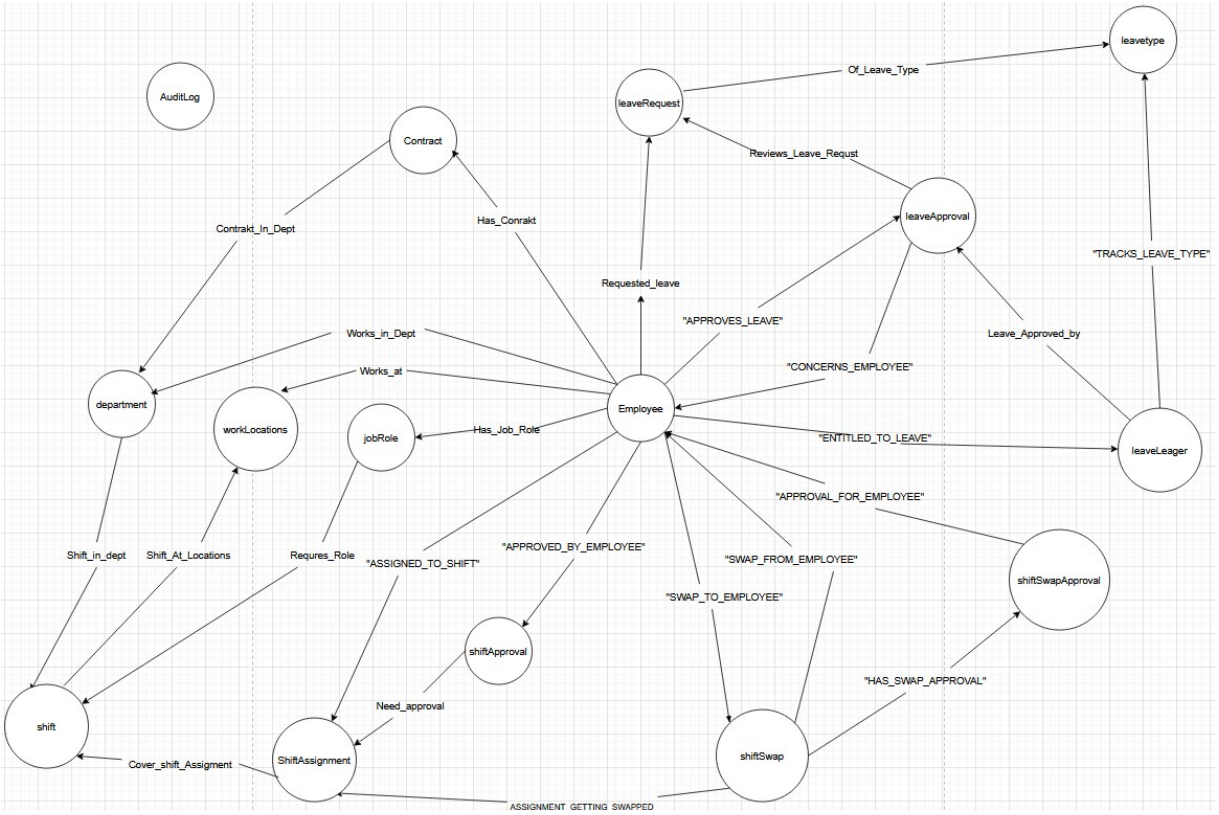
The graph database design is based on the domain model defined in the relational solution, where core entities are transformed into nodes and their interactions into relationships. This approach preserves the business logic while adapting it to a graph-oriented structure.

In this model, key domain entities such as Employee, Shift, Department, JobRole, WorkLocation, LeaveRequest, and ShiftSwap are represented as nodes. Relationships between these nodes reflect real-world interactions and workflows within the system. For example, an employee WORKS_AT_DEPT a specific location, HAS_JOB_ROLE in a job role, and can be ASSIGNED_TO_SHIFT a shift. Similarly, a shift SHIFT_AT_LOCATION connects it to a work location, and a leave request REVIEWS_LEAVE_REQUST links it to its approval process.

Special attention is given to the direction and meaning of relationships, as they define how the data is traversed in queries. Relationship names are chosen to be descriptive and aligned with the domain logic, ensuring that the graph remains intuitive and easy to understand. Additionally, unnecessary nodes are avoided to keep the model efficient and focused on meaningful connections.

An important concept in graph databases is that relationships are first-class citizens, meaning they are stored explicitly and can contain their own semantics, rather than being inferred through foreign keys as in relational databases. This allows for more natural modeling of complex interactions such as approvals and shift swaps.

Overall, the design process follows a similar progression as relational modeling (conceptual → logical), but instead of tables and foreign keys, the structure is expressed through nodes and relationships. The graphical model below illustrates the structure of the database and the connections between the main entities.



4.3. Short descriptions of features like indexes, transactions, PKs, constraints, stored objects (if stored objects are not available, how were they replaced?)

Indexes

Indexes in graph databases are used to improve performance by enabling efficient lookup of specific nodes. Unlike relational databases, where indexes are heavily used for join operations, in graph databases indexes are primarily used to quickly locate a starting node for traversal.

In Neo4j, once the correct starting node is found (for example an *Employee* node with a specific *employeeId*), the database can efficiently traverse its relationships to retrieve connected data. This means that indexes are not used for traversing relationships, but only for identifying entry points into the graph.

Indexes are typically created on unique identifier properties, such as *employeeId* or *shiftId*, as these are commonly used in queries. For example, when retrieving a specific employee or shift, an index ensures that the node can be located without scanning the entire dataset.

In this project, indexes are applied to identifiers because most queries begin by locating a specific entity. For instance, when handling shift swaps, the system needs to quickly find employees and their assigned shifts in order to determine availability. By indexing properties such as *employeeId* and *shiftId*, the system can efficiently identify relevant nodes and then traverse the graph to find employees who are available for a shift swap.

Unlike relational databases, indexes are not automatically created for domain-specific properties in Neo4j, so they must be defined explicitly.

Transactions

Graph databases such as Neo4j support ACID transactions, ensuring reliable and consistent data operations. A transaction represents a group of operations executed as a single unit of work, where either all changes are committed or none are applied in case of failure. This is particularly important when working with interconnected data, as it prevents partial updates and maintains consistency across related nodes and relationships.

In this project, the graph database is used to support shift swap functionality, which involves multiple connected entities such as employees, shifts, and approvals. A shift swap is modeled explicitly using a *ShiftSwap* node, connected to the involved employees and the relevant shift assignment through relationships such as *FROM_EMPLOYEE*, *TO_EMPLOYEE*, and *FOR_SHIFT*. This allows the swap to be represented as a domain event rather than a simple data update.

The graph structure enables efficient evaluation of swap conditions by traversing relationships. For example, the system can quickly determine whether a target employee is available by navigating their existing shift assignments and checking for conflicts, without relying on complex join operations.

The approval process is modeled using *ShiftSwapApproval* nodes, linked to both the swap and the approving employee. This provides a clear and traceable structure for managing the state and history of each swap request.

While the graph database models and queries the relationships involved, transactional control is handled in the application layer. The backend ensures that validation rules are enforced and that swap operations are executed atomically, preventing inconsistent states.

Primary Keys and Constraints

Unlike relational databases, graph databases do not use traditional primary keys. Instead, unique identification of nodes is achieved through properties combined with constraints. In this project, each entity contains a unique identifier property, such as *employeeId* for the *Employee* node or *shiftId* for the *Shift* node.

To ensure data integrity and prevent duplicate nodes, uniqueness constraints are applied to these identifier properties. For example, a constraint on *Employee.employeeId* guarantees that each employee is represented only once in the graph. These constraints also automatically create indexes, which improve performance when locating starting nodes for queries.

This approach replaces the concept of primary keys in relational databases while still ensuring consistent and reliable data identification across the graph model.

Stored Objects and Business Logic

Unlike relational databases, Neo4j does not natively support traditional stored objects such as stored procedures, triggers, or events in the same way as SQL-based systems. As a result, database-related logic cannot be embedded directly inside the database.

Instead, the majority of business logic in this project is implemented in the application layer. This includes validation, relationship creation, and transactional operations. By handling logic in the application, the system maintains better control over data consistency and workflow execution.

Neo4j does support procedures and user-defined functions, but these are more limited compared to stored procedures in relational databases and are typically used for specialized operations rather than core business logic.

Constraints and Data Integrity

Constraints in Neo4j play an important role in maintaining data integrity, similar to constraints in relational databases. However, instead of enforcing relationships through foreign keys, graph databases rely on application logic and controlled data creation.

In this project, constraints are primarily used to enforce uniqueness of key properties, ensuring that no duplicate entities exist in the database.

Since graph databases do not enforce referential integrity in the same way as relational databases, it is the responsibility of the application layer to ensure that relationships are created correctly and that no orphaned nodes exist. This provides more flexibility, but also requires careful design and validation.

4.4. Description of the CRUD application for the graph database
5. Discussion: similarities and differences between used database types, how each handled: Data modeling, Transactions, Queries, Performance, Constraints & integrity, Which database fits the domain best — and why, Trade-offs and compromises
6. Reflection: What was difficult about: Modeling the same domain in three databases, implementing transactions, auditing, and security, What design mistakes were made and fixed, what would be done differently next time, what you learned about databases in practice.
7. Conclusion: short high-level summary of the project, results, main insights.
8

8. References: All major design choices, technical claims, and comparisons must be supported by references. Sources must be cited in the text and listed in the References section. References should prim